A LEARNING PROJECT · CUSTOM AMBIENT OCCLUSION UNREAL ENGINE 5.8

A custom ambient occlusion in a UE5 plugin

lit

A LEARNING PROJECT · CUSTOM AMBIENT OCCLUSION UNREAL ENGINE 5.8

GTDAO: a custom ambient occlusion in a UE5 plugin



lit ray traced sky visibility GTDAO

3 · WHAT AMBIENT OCCLUSION IS

Ambient Occlusion

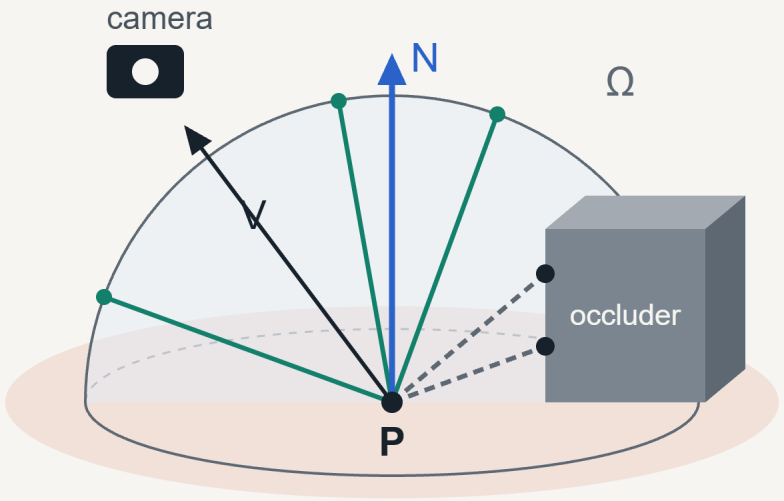


Informally

How much of the sky a point can see. The more its surroundings close in on it, the less ambient light reaches it: creases, corners and contact points get darker.

3 · WHAT AMBIENT OCCLUSION IS

Ambient Occlusion



$$AO(P) = \frac{1}{\pi} \int_{\Omega} (N \cdot \omega) \cdot IsVisible(\omega)\, d\omega$$

ω: a direction of the hemisphere Ω

1 · Lumen Global Illumination + Short Range Ambient Occlusion

the global illumination already occludes; Short Range Ambient Occlusion adds the fine contacts

2 · Screen Space Global Illumination

Lumen off; occlusion comes from the screen space global illumination

3 · No dynamic Global Illumination, or a plugin

RTAO Ray Traced Ambient Occlusion

SSAO Screen Space Ambient Occlusion, the default

GTAO Ground Truth Ambient Occlusion

And ...

DFAO Distance Field Ambient Occlusion: sky light, not with Lumen

Material Ambient Occlusion: baked per mesh

A LEARNING PROJECT · UNREAL ENGINE 5.8

The repository

The GI plugin hook

UE video

Project Settings → Dynamic Global Illumination Method = Plugin · the plugin code that registers the delegate

4 · HOW MANY AOS UNREAL HAS

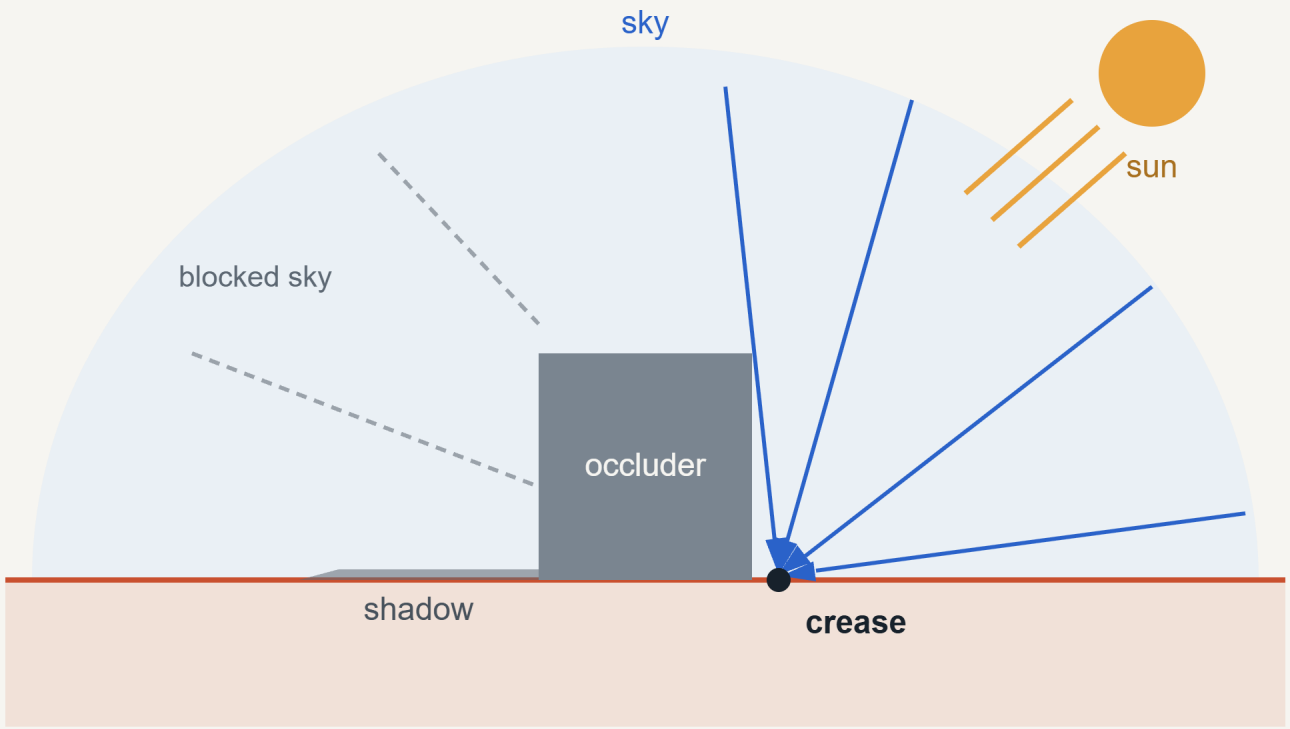
Getting the screen AO into the scene

UE video

Lumen GI off · Sky Light (movable): Cast Ray Traced Shadows off · ShowFlag.DistanceFieldAO 0 · toggle the AO and see it in the lit scene

3 · WHAT AMBIENT OCCLUSION IS

What AO darkens, and what it does not



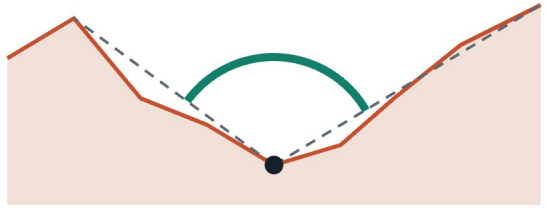
Direct light: its occlusion is the **shadow**, not AO.

Sky light: the crease sees less sky → **AO** darkens it.

In UE the AO darkens only the movable Sky Light diffuse, reflections and baked lighting.

5 · WHAT I DID

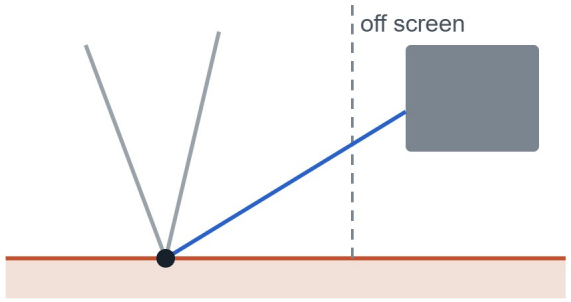
GTAO + distance field



Screen-space horizon march

GTAO style · contacts and creases · only on screen, within the radius

×



Distance field samples

large, off-screen and hidden occluders

=

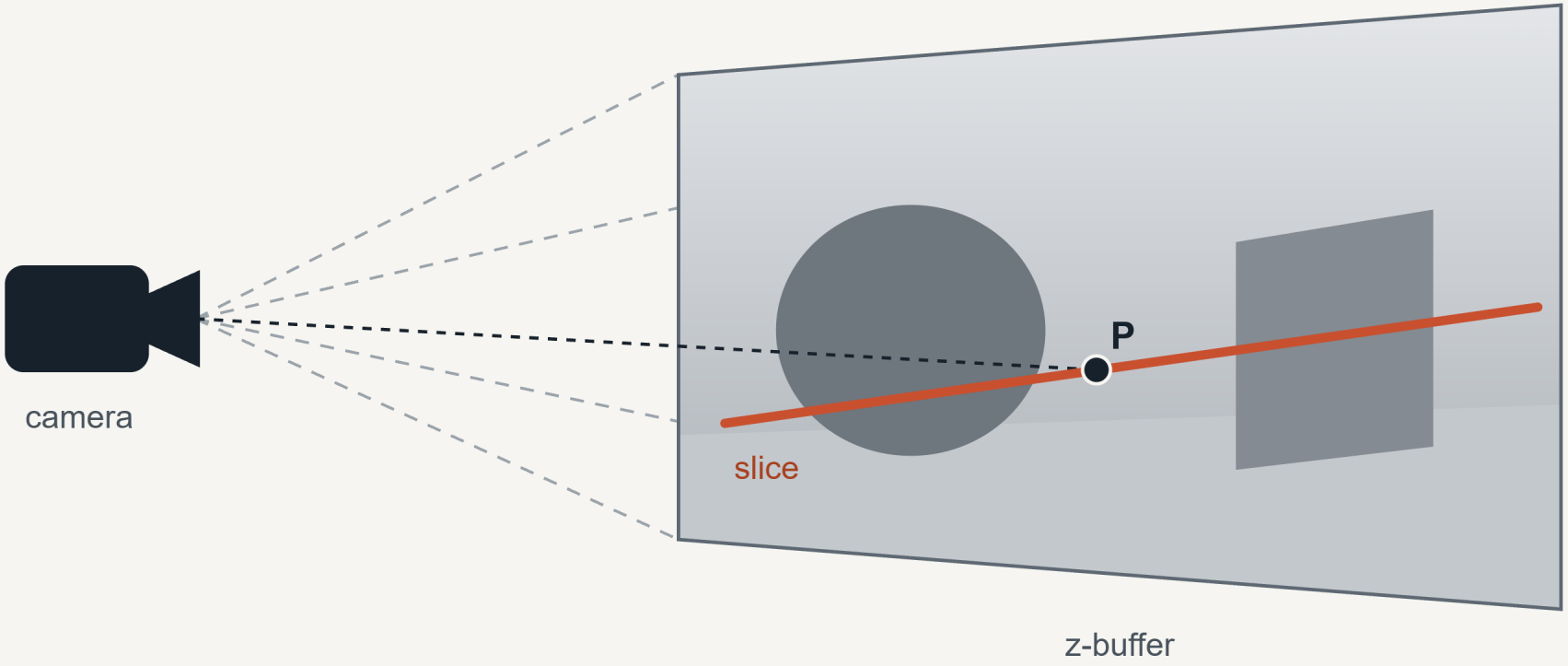


GTDAO

written into UE’s AO texture

HORIZON-BASED AMBIENT OCCLUSION

Approximating AO in screen space



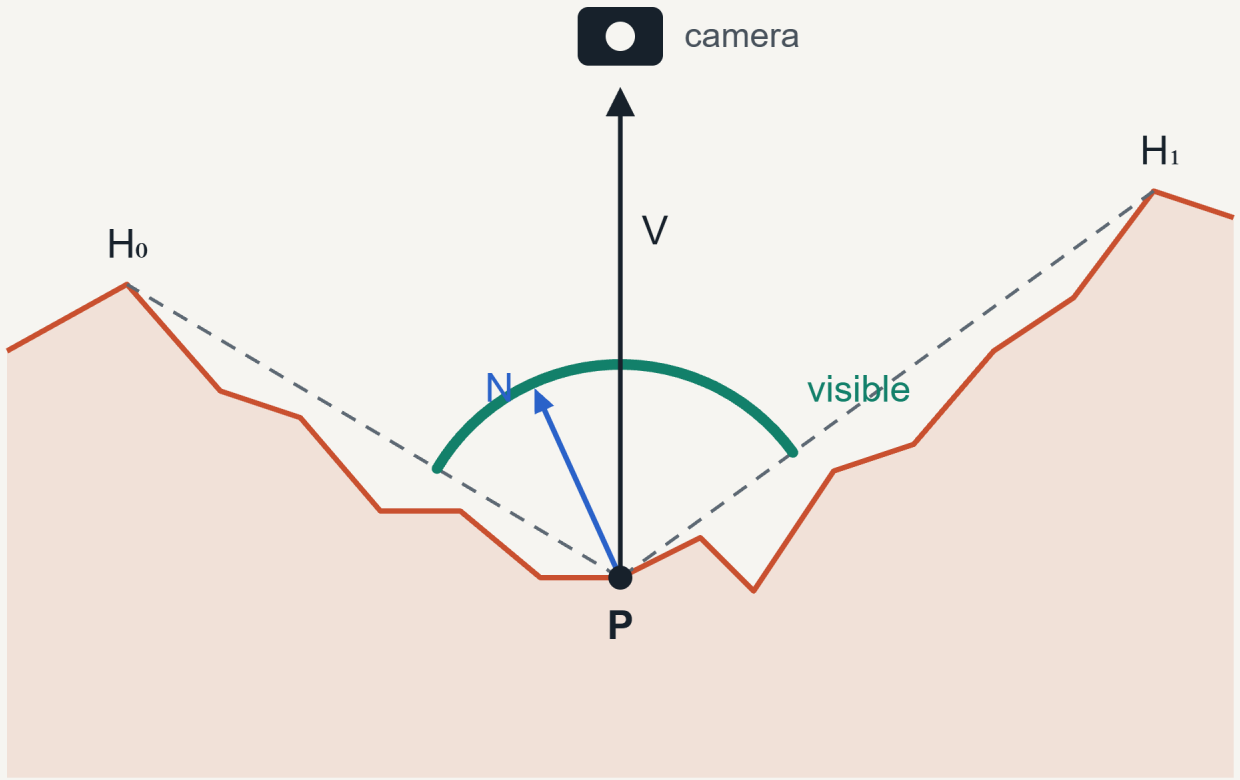
— **slice** a line on the screen through P

● **P** the pixel being shaded

z-buffer the depth of every pixel

HORIZON-BASED AMBIENT OCCLUSION

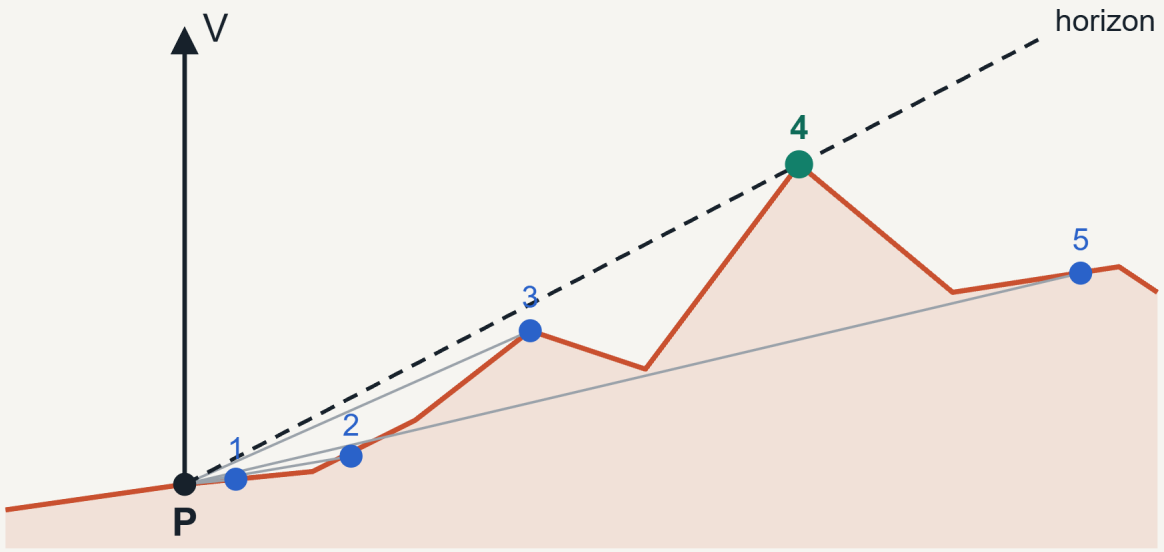
The slice, seen from the side



- depth buffer along the slice
- **P** the pixel being shaded
- **N** the normal, from the GBuffer
- **V** from P to the camera
- **H_0, H_1** horizons: highest sample on each side
- visible directions, between the horizons

COMPUTEHORIZONAO · MARCHING ONE SIDE

Each step raises the horizon, or not



sample	1	2	3	4	5
CosH → MaxCos	0.10 → 0.10	0.17 → 0.17	0.41 → 0.41	**0.46 → 0.46**	0.23 → 0.46

1. Rebuild the sample: Ps from its pixel and depth.

2. Direction from P: Delta = Ps − P.

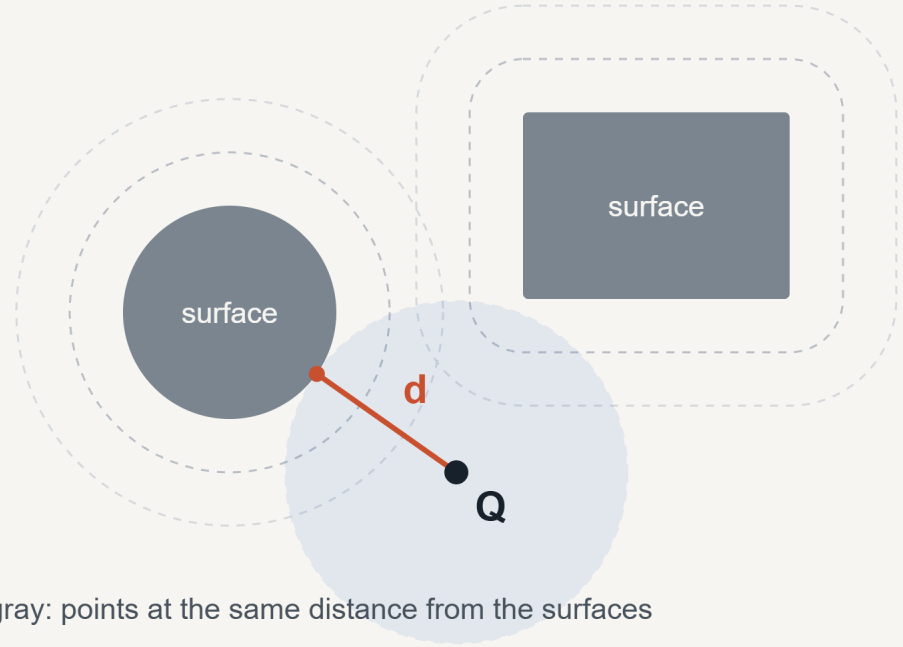
3. How high it is: CosH = dot(Delta, V) / |Delta|.

4. Keep the highest: MaxCos = max(MaxCos, CosH).

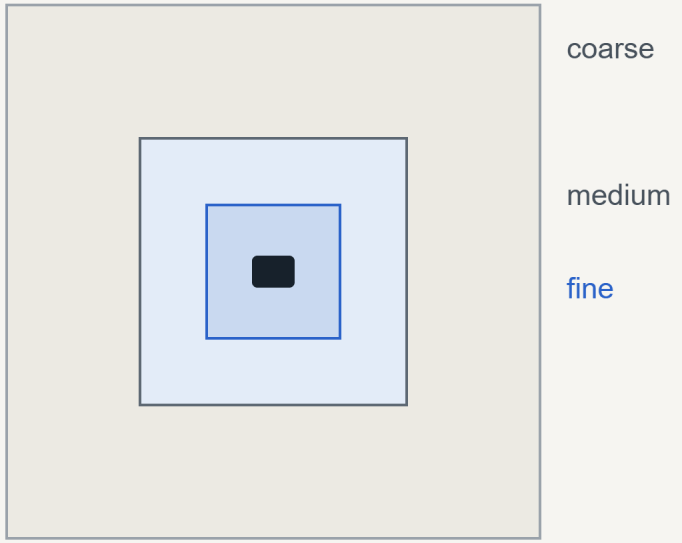
Falloff and thickness adjust step 4: next.

7 · THE DISTANCE FIELD

What a distance field is

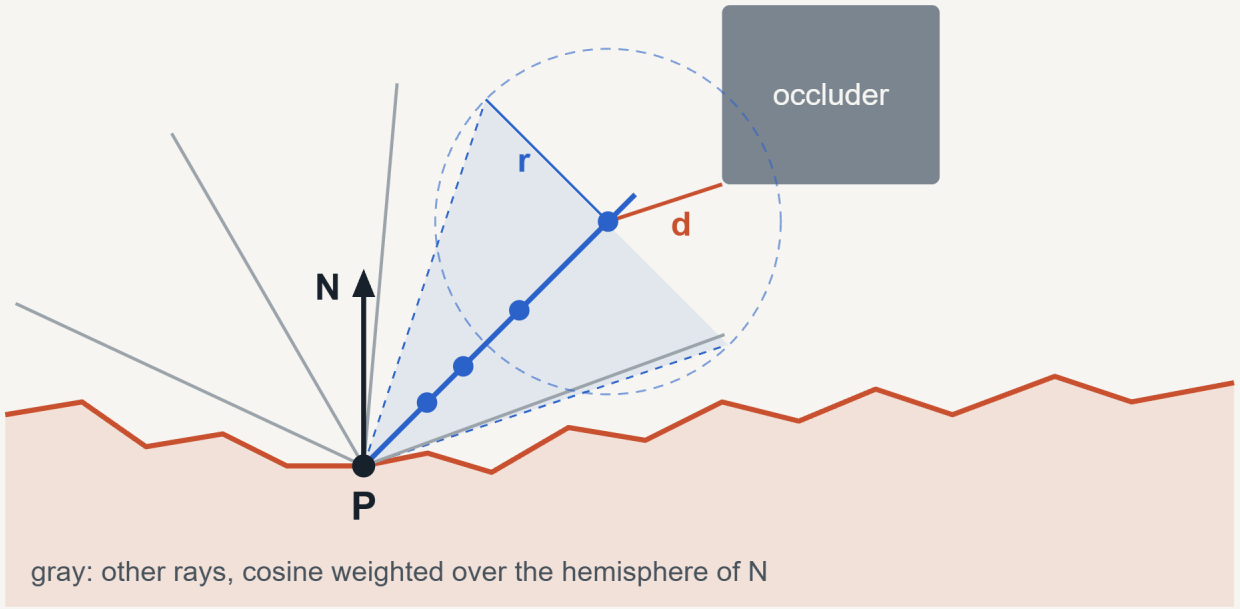


dashed gray: points at the same distance from the surfaces



camera at the center

Distance field rays: cone samples



1. A few rays over the whole hemisphere of N, cosine weighted.

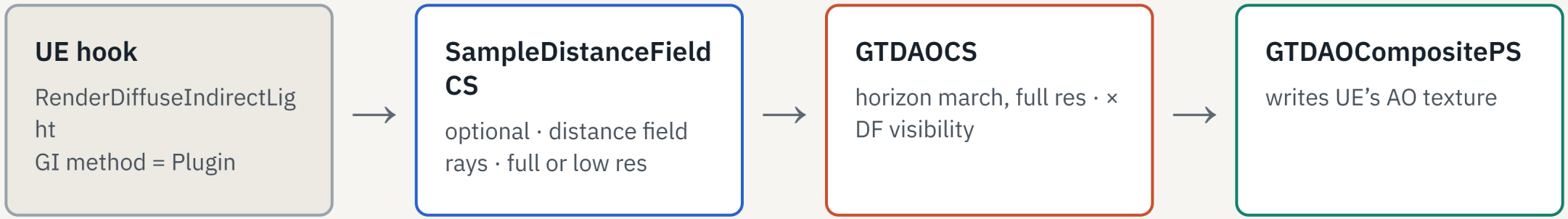
2. No march: 4 samples per ray, geometric spacing.

3. Free part of the cone: saturate(d / r). Ray = the smallest.

4. Average of the rays × screen-space AO.

2 · THE GI PLUGIN HOOK

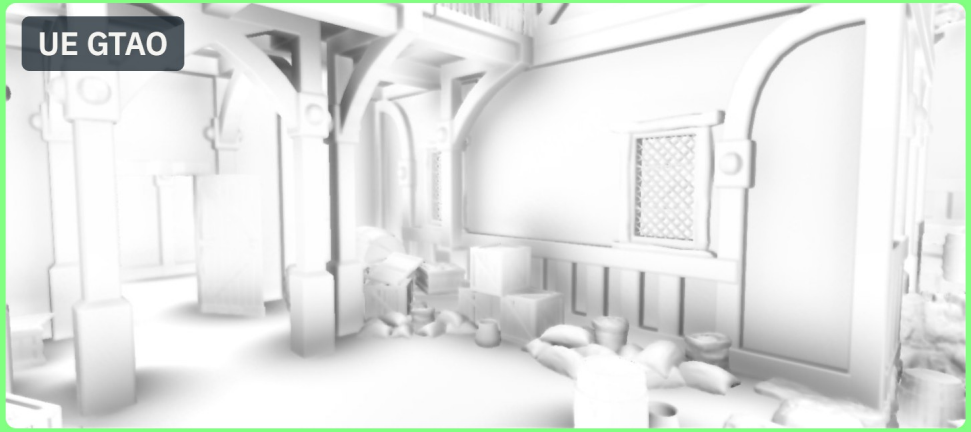
The plugin: one hook, three passes



View B1









THE GOAL

The integral to compute

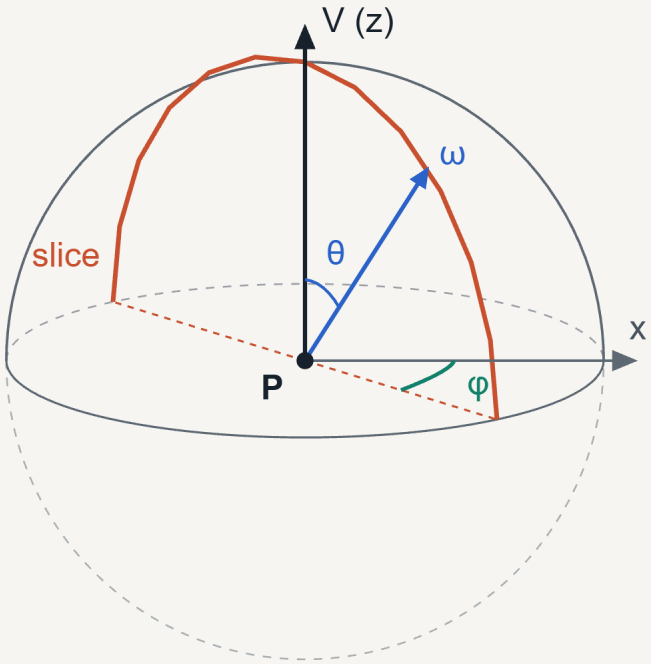
$$A = \frac{1}{\pi} \int_{\Omega} V(\omega) \cdot (N \cdot \omega)\, d\omega$$

Uniform ambient light reaching P, cosine weighted, normalized to [0, 1]

On the whiteboard	Correct	Why
∫ from −π to π	θ ∈ [−π/2, π/2]	In a slice the hemisphere is a half circle: π in total
IsOccluded(d)	V(ω) = 1 − IsOccluded(ω)	We integrate the light that arrives, not the blocked one
no normalization	1/π	Fully visible gives π: dividing, A ∈ [0, 1]
missing measure	dω (solid angle)	We integrate over directions: each one weighs by its area

STEP 1 · COORDINATES

Spherical coordinates with polar axis V



$$\omega(\theta, \varphi) = (\sin\theta\cos\varphi, \sin\theta\sin\varphi, \cos\theta)$$

θ is the angle from V; **φ** is the rotation around V.

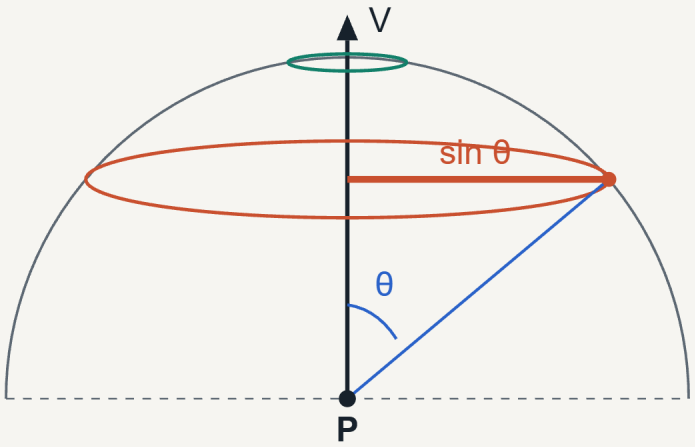
Why V: at a fixed φ the directions lie in a plane containing V, and on screen that plane is a line. It is the slice to march.

GTAO reparametrization: $\varphi \in [0, \pi)$ picks the line; a signed $\theta \in [-\pi/2, \pi/2]$ picks the side ($\theta < 0$ = side $\varphi + \pi$).

Careful: this is the hemisphere around V, not around N. The one around N comes back with the clamp (step 5).

STEP 2 · SOLID ANGLE

The area of a small dθ × dφ patch on the sphere



Moving θ by dθ slides along a meridian (radius 1): length dθ.

Moving φ by dφ turns along a parallel of radius sin θ: length sin θ · dφ.

$$d\omega = |\sin\theta|\, d\theta\, d\varphi$$

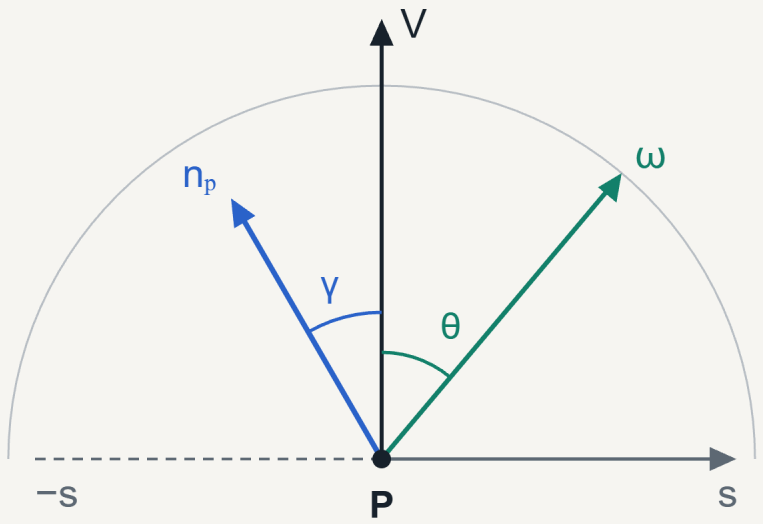
The absolute value is needed because θ is signed: the radius of the parallel is a distance.

Check: ∫ over φ ∈ [0, π) and θ ∈ [−π/2, π/2] of |sin θ| = π · 2 = 2π, the area of the hemisphere.

The slices squeeze around V like orange segments at the pole: there each one owns little area.

STEP 3 · THE COSINE

Rewriting N·ω to integrate it easily



$$\omega(\theta) = \sin\theta \cdot s + \cos\theta \cdot V$$

$$N\cdot\omega = \sin\theta\,(N\cdot s) + \cos\theta\,(N\cdot V)$$

$$n_p = (N\cdot s)\,s + (N\cdot V)\,V = |n_p|\,(\sin\gamma \cdot s + \cos\gamma \cdot V)$$

$$\Rightarrow N\cdot s = |n_p|\sin\gamma,\ N\cdot V = |n_p|\cos\gamma$$

$$|n_p| = \sqrt{((N\cdot s)^2 + (N\cdot V)^2)},\ \gamma = \operatorname{atan2}(N\cdot s, N\cdot V)$$

$$N\cdot\omega = |n_p|\,(\cos\theta\cos\gamma + \sin\theta\sin\gamma)$$ substituting into $N\cdot\omega$

$$\Rightarrow N\cdot\omega = |n_p|\cos(\theta - \gamma)$$ cosine of a difference

STEP 4 · ASSEMBLY AND VISIBILITY

The integral of one slice

Substituting $\omega \to (\theta, \varphi)$, $d\omega \to |\sin\theta|\, d\theta\, d\varphi$ and $N\cdot\omega \to |n_p| \cos(\theta - \gamma)$; $|n_p|$ and γ depend only on φ, so $|n_p|$ comes out of the inner integral.

$$A = \frac{1}{\pi} \int_0^{\pi} |n_p| \left[\int_{-\pi/2}^{\pi/2} V(\theta, \varphi) \cos(\theta - \gamma) |\sin\theta|\, d\theta \right] d\varphi$$

Heightfield: V = 1 only between the horizons, 0 below. V disappears and only the bounds remain; the bracket becomes

$$\hat{a} = \int_{h_0}^{h_1} \cos(\theta - \gamma) |\sin\theta|\, d\theta$$

How h_0 and h_1 are found

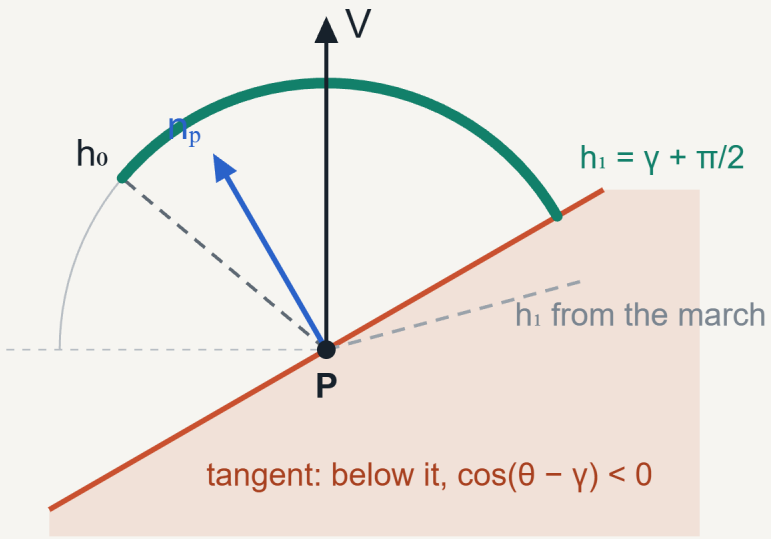
For each sample S: $\omega_S = (S - P) / |S - P|$ and $\cos\theta_S = \omega_S \cdot V$. The horizon is the sample closest to V: $h_1 = +\arccos(\max \omega_S \cdot V)$ on side s, $h_0 = -\arccos(\max \omega_S \cdot V)$ on side −s.

What is left

The whole scene ended up in two numbers. θ is not computed: it runs inside the integral. The march keeps only the maximum cosine, and takes the arccos once at the end.

STEP 5 · CLAMP

Horizons limited to the hemisphere of N



cos(θ − γ) < 0 means ω is more than 90° from n_p: behind the surface. Left in, the negative cosine would subtract light.

In the slice the boundary is the **tangent**, 90° from the projected normal: at the angles γ − π/2 and γ + π/2.

$$h_0 \leftarrow \max(h_0, \gamma - \pi/2)$$
$$h_1 \leftarrow \min(h_1, \gamma + \pi/2)$$

After the clamp cos(θ − γ) ≥ 0 over all of $[h_0, h_1]$: max(0, N·ω) is no longer needed.

STEP 6 · SOLUTION

The integral in closed form

1 · Split at θ = 0
drop the absolute value

$$\hat{a} = \int_{h_0}^{0} \cos(\theta - \gamma)(-\sin\theta)\, d\theta + \int_{0}^{h_1} \cos(\theta - \gamma)\sin\theta\, d\theta$$

2 · Product → sum

$$\cos(\theta - \gamma)\sin\theta = \tfrac{1}{2}\left[\sin(2\theta - \gamma) + \sin\gamma\right]$$

3 · Antiderivative

$$\tfrac{1}{2}\left[-\tfrac{1}{2}\cos(2\theta - \gamma) + \theta\sin\gamma\right]$$

4 · One side, signed h
same form for h_0 and h_1

$$\tfrac{1}{4}\left(-\cos(2h - \gamma) + \cos\gamma + 2h\sin\gamma\right)$$

5 · Sum and check

$$\hat{a} = \text{side}(h_0) + \text{side}(h_1);\ \gamma = 0,\ h = \pm\pi/2 \Rightarrow \tfrac{1}{2} + \tfrac{1}{2} = 1$$

On side −s the sign of −sin θ and the swapped bounds cancel out: this is why both sides have the same formula.

STEP 7 · ALL THE SLICES

From one slice to the pixel’s AO

$$A = \frac{1}{\pi} \int_0^{\pi} |n_p|\, \hat{a}(h_0, h_1, \gamma)\, d\varphi$$

Normalization: fully visible with N = V gives $\hat{a} = 1$ for every slice, so $\int = \pi$. It is the classic $\int (N \cdot \omega)\, d\omega = \pi$ over the hemisphere.

Discretization: numSlices angles φ_i, each worth $\Delta\varphi = \pi$ / numSlices. The two π cancel out:

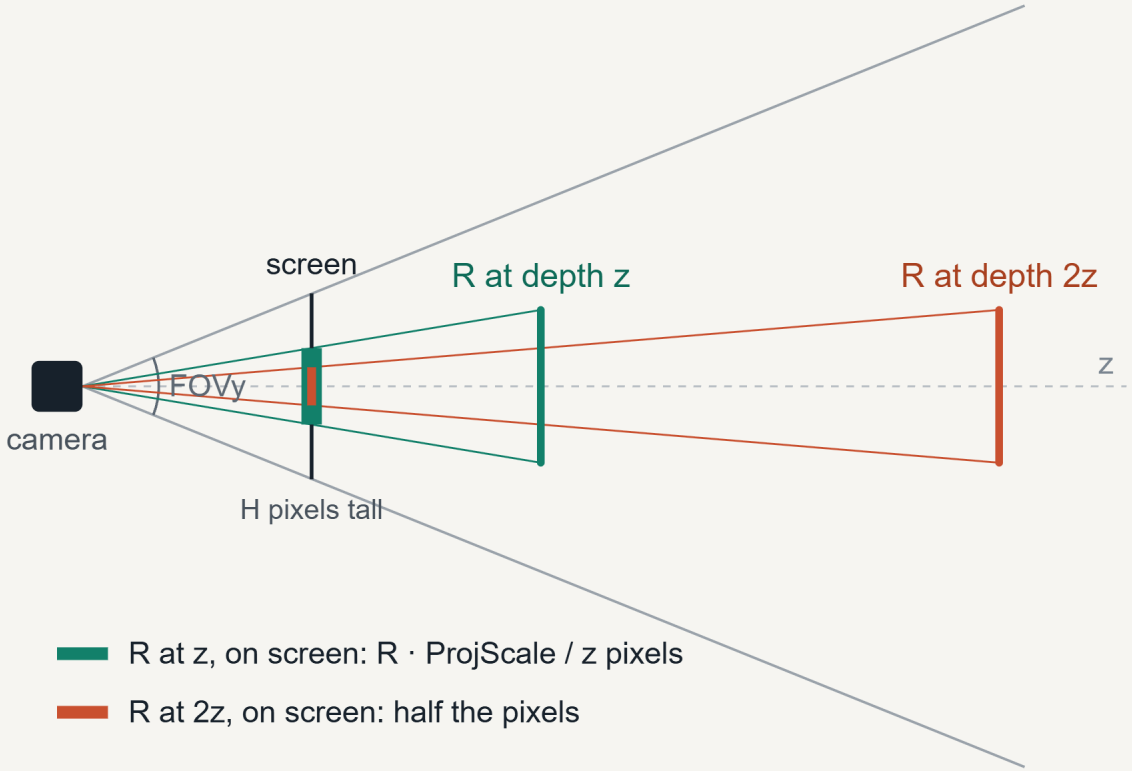
$$A \approx \frac{1}{\text{numSlices}} \sum_i |n_p|_i\, \hat{a}_i$$

For each pixel

1. φ_i with per-pixel noise
2. s orthogonal to V, then $|n_p|$ and γ
3. march: h_0 and h_1 from max $\omega_S \cdot V$
4. clamp with $\gamma \pm \pi/2$
5. $\hat{a}$ with the closed form
6. accumulate $|n_p| \cdot \hat{a}$, divide by numSlices

COMPUTEHORIZONAO · SETUP

Compute the marching radius in pixels



Compute the marching radius in pixels

The AO radius is a parameter in world units (UU): we convert it into pixels to get the **marching radius**.

```
ProjScale = 0.5 * View.ViewSizeAndInvSize.y * View.ViewToClip[1][1]
```

- View.ViewToClip[1][1] = 1/tan(FOVy/2): maps length / z to NDC
- 0.5 * View.ViewSizeAndInvSize.y (the viewport height in pixels): maps NDC to pixels

Result: the pixels covered by a camera-space length of 1 at z = 1.

```
RadiusPixels = min(Radius * ProjScale / P.z, MaxRadiusPixels)
```

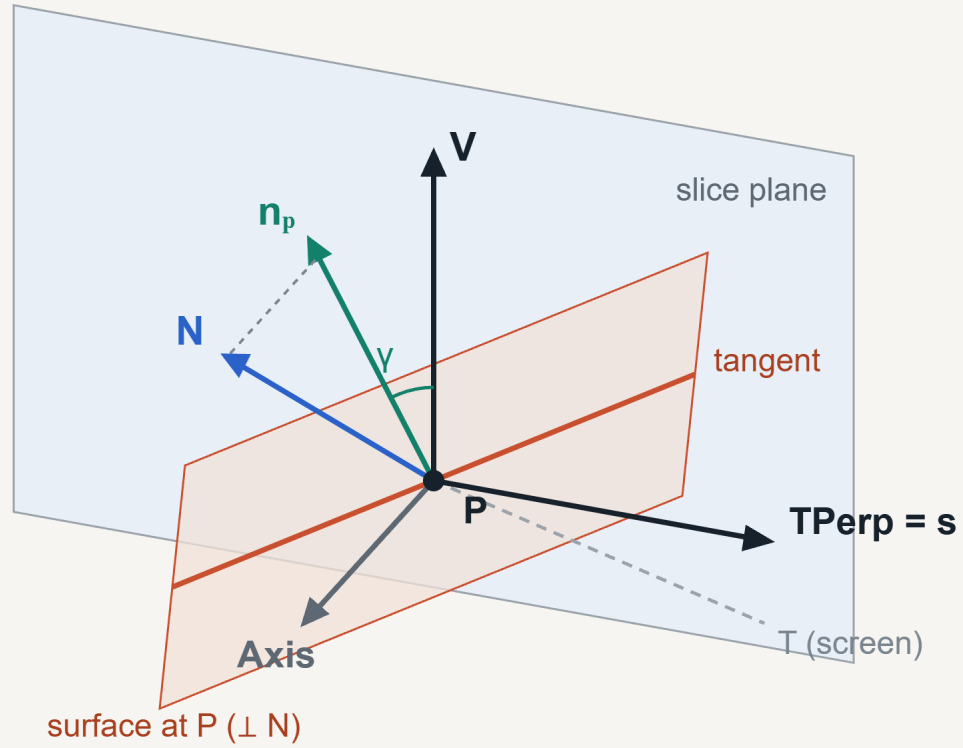
- ProjScale / P.z: the pixels covered by a camera-space length of 1 at depth P.z
- Radius * ...: the pixels covered by the radius (world units) at that depth
- min(..., MaxRadiusPixels): a cap for surfaces very close to the camera

```
if (RadiusPixels < 1.0) return 1.0;
```

Too far: the marching radius is smaller than a pixel, nothing to march. Unoccluded.

COMPUTEHORIZONAO · PER SLICE

The slice frame around P, and the normal



Frame: TPerp (x) and V (y) span the slice plane; Axis is perpendicular to it.

The frame sits on the surface: P lies on it, and the slice cuts it along the **tangent**.

T is on the screen plane, but not orthogonal to V: it only helps to build the frame.

n_p: N without its Axis component, the part of N that lies in the slice.

$|n_p|$ is the slice weight; **γ** is the angle of n_p from V.

COMPUTEHORIZONAO · PER SLICE

Slice frame and projected normal

```
// Create a local reference system for the slice around the point P:
// TPerp: slice direction orthogonal to V
// V: direction from the pixel P toward the camera
// Axis: perpendicular to V and TPerp
const float3 T = float3(Dir, 0);
const float3 Axis = normalize(cross(T, V));
const float3 TPerp = cross(V, Axis);

// Normal projected onto the slice plane
// 1. Remove the normal's component not on the slice plane
const float3 Np = N - Axis * dot(N, Axis);
const float Len = length(Np); // |Np|: the slice weight, used later
// 2. Signed angle between Np and V. max(Len, 1e-4) avoids a division by zero;
// acos returns only the unsigned angle [0, π], sign(dot(Np, TPerp)) gives its side
const float Gamma = sign(dot(Np, TPerp))
    * acos(clamp(dot(Np, V) / max(Len, 1e-4), -1.0, 1.0));
```

STEP 8 · IN CODE

IntegrateArc: one side of the slice

```
// visible arc from V (θ = 0) to the horizon h, n = γ
float IntegrateArc(float h, float n)
{
    return 0.25 * (-cos(2.0 * h - n) + cos(n)
                + 2.0 * h * sin(n));
}

// for each slice (h0 < 0 < h1, already clamped)
float a = IntegrateArc(h0, gamma) + IntegrateArc(h1, gamma);
visibility += lenNp * a;
// at the end
visibility /= numSlices;
```

It is the step 6 formula for one side: the weights $\cos(\theta - \gamma)$ and $|\sin\theta|$ are already inside.

Check 1

h0 must arrive **negative**, otherwise $2h \cdot \sin(n)$ has the wrong sign.

Check 2

The clamp with $\gamma \pm \pi/2$ must be done **before**: the function does not do it.

7 · WHAT WE ADD ON TOP

MarchSample: HZB and invalid samples

```
// one step, one side of the slice
if (Distance ⩾ 4.0) // far: HZB
{
    Mip = min(int(log2(Distance * 0.25)), 2);
    Texel = int2((SamplePos - ViewMin) * 0.5) >> Mip;
    DeviceZ = HZB.Load(int3(Texel, Mip)).r; // furthest z
    SamplePos = ViewMin + (Texel + 0.5) * (2 << Mip);
}
else DeviceZ = SceneDepth.Load(int3(SamplePos, 0)).r;

// off screen or sky: leave the horizon as it is
Falloff = (bOnScreen && DeviceZ > 0.0)
        ? saturate(DeltaLen / Radius * 2.0 - 1.0) : 1.0;
CosH = lerp(CosH, MaxCos, Falloff);
```

HZB: Hierarchical Z-Buffer, furthest depth per block.

4–8 px → mip 0 · 8–16 px → mip 1 · then mip 2.

We march pixels, but land on mip texels.

Invalid sample: full falloff, horizon unchanged.

7 · WHAT WE ADD ON TOP

SampleDistanceFieldCS

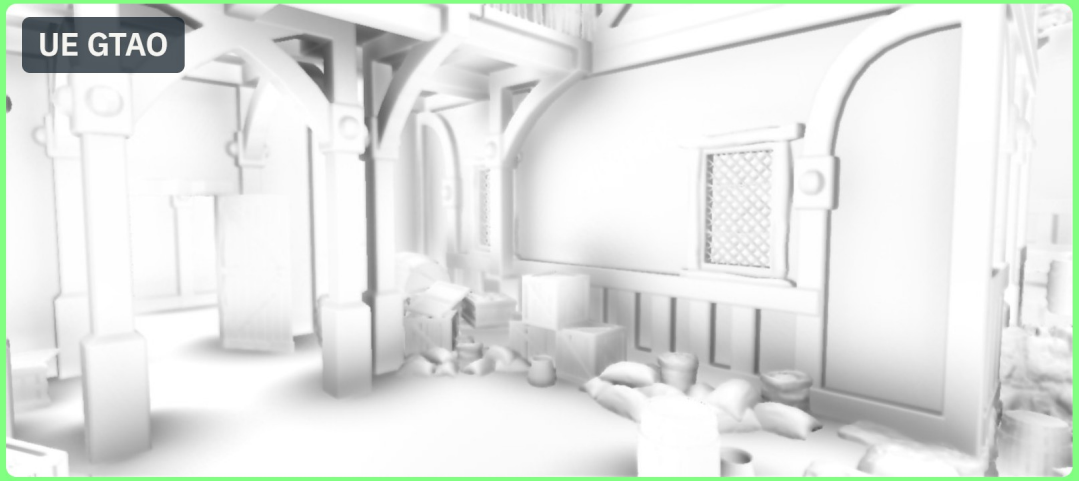
```
RayStart = SurfacePos + N * 2.0 * VoxelExtent;
for (Ray = 0; Ray < DFRaysPerTexel; ++Ray)
{
    RayDir = TangentToWorld(CosineSampleHemisphere(E), N);
    RayVis = 1.0;
    for (i = 0; i < 4; ++i)
    {
        t = MinDistance * exp2(LogRange * (i + Jitter) / 4);
        d = GlobalSDF(RayStart + RayDir * t);
        r = min(t * DF_CONE_TAN, 0.9 * Band);
        RayVis = min(RayVis, saturate(d / r));
    }
    Visibility += RayVis;
}
Visibility /= DFRaysPerTexel; // later: AO *= Visibility
```

DF_CONE_TAN = 0.5

MinDistance 50 cm · MaxDistance 300 cm

Low res: one ray per N×N pixels, bilinear upsample.

UE GTAO vs our AO





Results: one scene, one PC

Error vs ray traced sky visibility

	MAE	GPU ms
UE GTAO	0.551	0.121
GTDAO, DF full res	0.347	0.224
GTDAO, DF 1/2 res	0.317	0.193
GTDAO, DF 1/3 res	0.312	0.183

Cost vs DFAO

	GPU ms
UE GTAO alone	0.121
UE GTAO + DFAO	0.486
UE SSAO + DFAO	~0.54
GTDAO, DF full res	0.225
GTDAO, DF 1/3 res	0.184

PL_Medieval_Docks · 4 views · 1395×619 · RTX 2070 laptop · radius 141 cm · 2 slices × 8 steps · DF 0.5–3 m · no denoiser

View B4



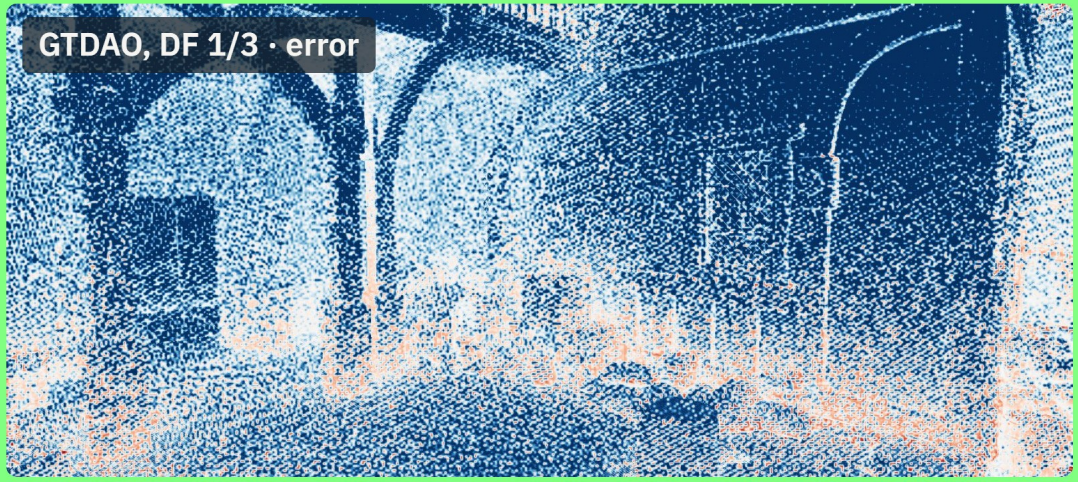






Error vs reference, view B1





Installing the plugin

1	A C++ project, UE 5.8 (launcher is enough)
2	git clone into <YourProject>/Plugins/GTDAO
3	Open the project: build the GTDAO module
4	Project Settings → Dynamic Global Illumination Method = Plugin

Console variables

Our AO alone

`r.DynamicGlobalIlluminationMethod 3`

`r.AmbientOcclusion.Method 0`

`r.AmbientOcclusionLevels 0`

`r.GTDAO.AO 1`

Distance field rays

`r.GenerateMeshDistanceFields 1` · restart

`r.DistanceFieldAO 1`

`ShowFlag.DistanceFieldAO 0` · hide DFAO

`r.AOGlobalDistanceField.DetailedNecessityCheck 0`

`r.GTDAO.DF 1`

Lit image

movable Sky Light · Cast Ray Traced Shadows off

Tuning

`r.GTDAO.Radius` · Slices · Steps · 0 / 2 / 8

`r.GTDAO.DF.Rays` · MinDistance · MaxDistance · 1 / 50 / 300

Limits and next steps

Denoiser	none yet: spatial (à-trous) + temporal, or UE ScreenSpaceDenoise
March cost	~2× UE GTAO march; lower res + depth-aware upsample
DF upsample	bleeds across depth edges
Thickness	still infinite; visibility bitmask
Bias	brighter than sky visibility near big occluders

Prior work and thanks

GTAO	Jimenez, Wu, Pesce, Jarabo · SIGGRAPH 2016 · UE PostProcessAmbientOcclusion.usf
UE DFAO · Lumen	min(DFAO, SSAO) · cone samples · screen + distance field tracing
TurboGI devlog	“Global Illumination for Poor People”
GI plugin hook	Tatsuhiro Tanaka (Qiita, 2024) · Dmitry Karpukhin, @skx_doom (80 Level, 2025)
Repository	github.com/salvatorespoto/GTDAO